

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет ИТМО»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 1

Дисциплина «Информационная безопасность»
Разработка защищённого REST API с интеграцией в CI/CD

Выполнил: Миронин Картюем Дмитриевич

Группа: П 3431

Санкт-Петербург
2026

Отчёт по лабораторной работе № 1

Разработка защищённого REST API с интеграцией в CI/CD

1. Цель работы

Получить практический опыт разработки защищённого backend-приложения, реализовать меры защиты от распространённых угроз OWASP и встроить проверки безопасности в процесс непрерывной интеграции.

2. Используемый стек

- JavaScript и Node.js 24;
- Express — HTTP API;
- SQLite (node:sqlite) — хранение данных;
- bcryptjs — хэширование паролей;
- jsonwebtoken — создание и проверка JWT;
- Helmet — безопасные HTTP-заголовки;
- ESLint Security — SAST;
- npm audit — SCA;
- GitHub Actions — CI/CD;
- node:test и Supertest — интеграционные тесты.

Стек выбран так, чтобы приложение оставалось небольшим и понятным. В проекте нет отдельного ORM: встроенный драйвер SQLite позволяет явно показать параметризацию запросов.

3. Структура и запуск проекта

Основные части проекта:

- src/app.js — сборка Express-приложения и маршруты;
- src/database.js — подключение и схема SQLite;
- src/repositories/ — запросы к данным;
- src/services/auth-service.js — проверка пароля и операции с JWT;
- src/middleware/authenticate.js — защита маршрутов;
- src/security/output.js — экранирование выходных строк;
- test/api.test.js — интеграционные тесты;
- .github/workflows/ci.yml — CI pipeline.

Команды запуска:

```
npm ci
cp .env.example .env
npm start
```

Приложение читает настройки из окружения. Секрет JWT должен содержать не менее 32 символов. Учебный пользователь создаётся при первом запуске, причём в базе сохраняется только bcrypt-хэш его пароля.

4. Реализованное API

4.1 POST /auth/login

Метод принимает login и password. При корректных данных возвращает сведения о пользователе и подписанный JWT. При ошибке возвращается HTTP 401. Тело запроса проверяется до обращения к базе, а число попыток ограничено.

```
POST /auth/login
Content-Type: application/json

{"login":"student","password":"ChangeMe123!"}
```

4.2 GET /api/data

Метод возвращает список постов. Перед обработкой middleware требует заголовок Authorization: Bearer <token>. Без токена, с изменённым или просроченным токеном сервер отвечает HTTP 401.

4.3 POST /api/posts

Самостоятельно разработанный третий метод создаёт пост. Заголовок обязателен и ограничен 120 символами, содержимое — 2000 символами. Автор берётся из проверенного JWT, а не из пользовательского тела запроса.

5. Реализованные меры защиты

5.1 Защита от SQL Injection

Все обращения к базе используют prepared statements. Значения передаются драйверу отдельно от текста SQL:

```
SELECT id, login, password_hash FROM users WHERE login = ?
```

Строка вроде student' OR 1=1 -- остаётся обычным значением параметра и не изменяет структуру запроса. Конкатенация пользовательского ввода с SQL нигде не используется. Этот сценарий также закреплён интеграционным тестом.

5.2 Защита от XSS

API возвращает JSON и не формирует HTML. Дополнительно поля title, content и author экранируются непосредственно перед включением в ответ. Например, <script> превращается в <script>. Данные хранятся без потери исходного содержимого, а безопасное представление формируется на границе API.

Helmet добавляет защитные заголовки, включая Content-Security-Policy и X-Content-Type-Options: nosniff.

5.3 Защита аутентификации

- bcrypt с cost factor 12 делает перебор паролей вычислительно дорогим;
- открытые пароли не сохраняются и не возвращаются;
- JWT подписывается только алгоритмом HS256 и имеет срок действия;
- при проверке явно разрешён только HS256;
- идентификатор автора берётся из проверенного поля sub JWT;
- все маршруты /api/* защищены единым middleware;
- endpoint входа ограничен десятью запросами в минуту с одного адреса;
- размер JSON-тела ограничен 10 КБ.

6. CI/CD и проверки безопасности

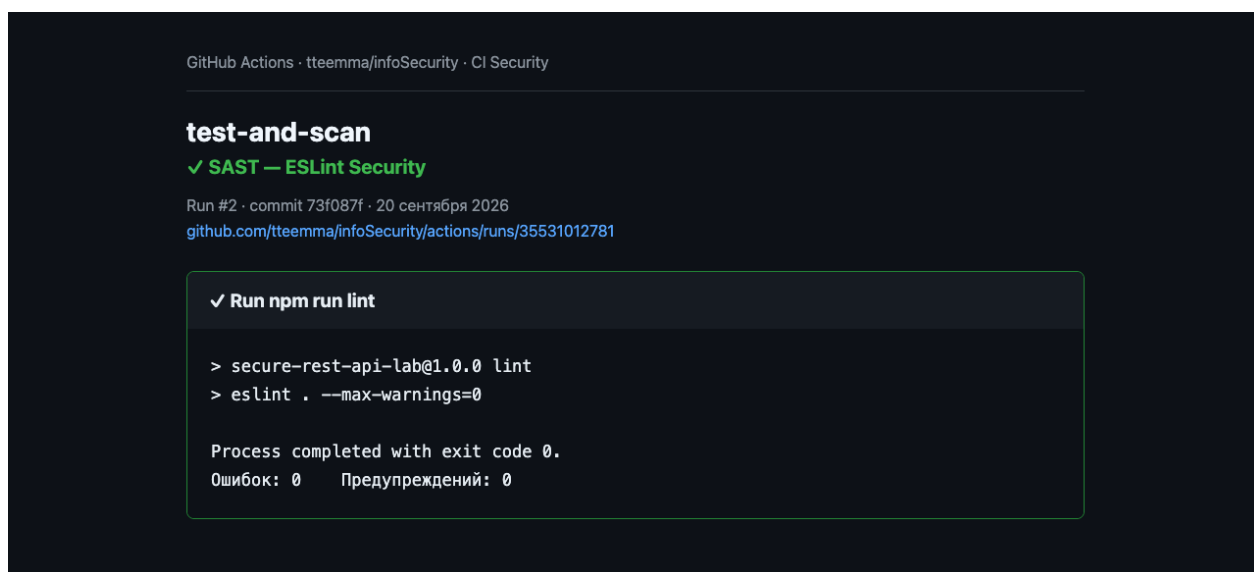
Workflow запускается при каждом push и pull_request и имеет минимальное разрешение contents: read. Последовательность шагов:

1. получение кода;
2. установка Node.js 24 и зависимостей через npm ci;
3. интеграционные тесты;
4. SAST с ESLint Security;
5. SCA с npm audit.

6.1 Результат локального SAST

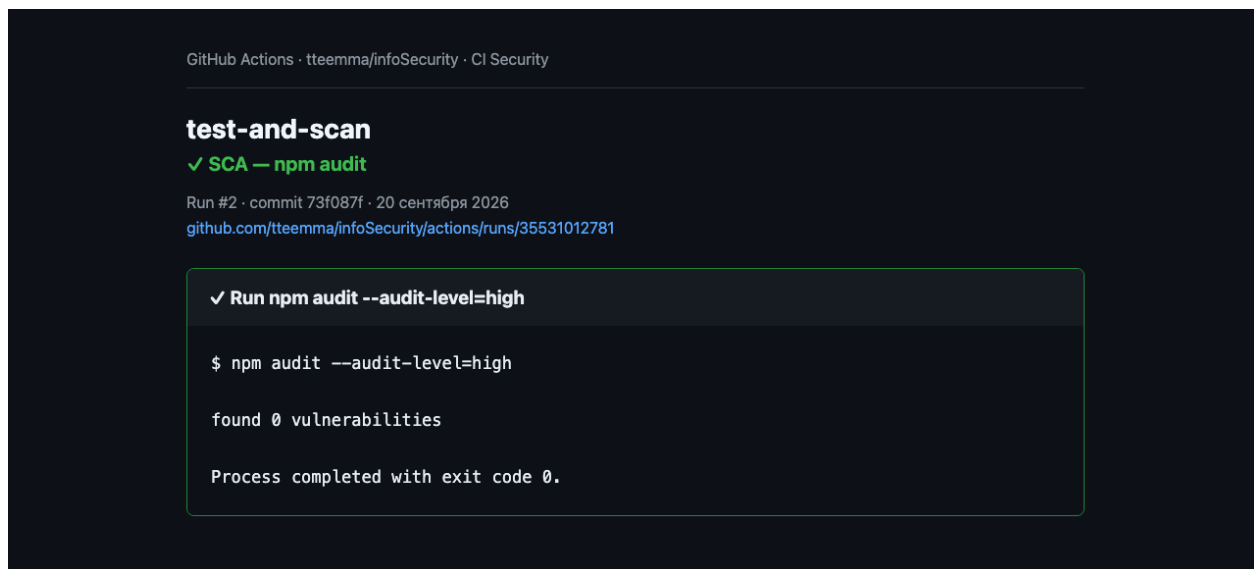
```
$ npm run lint
> eslint . --max-warnings=0
```

Process exited with code 0. Предупреждений и ошибок не найдено.

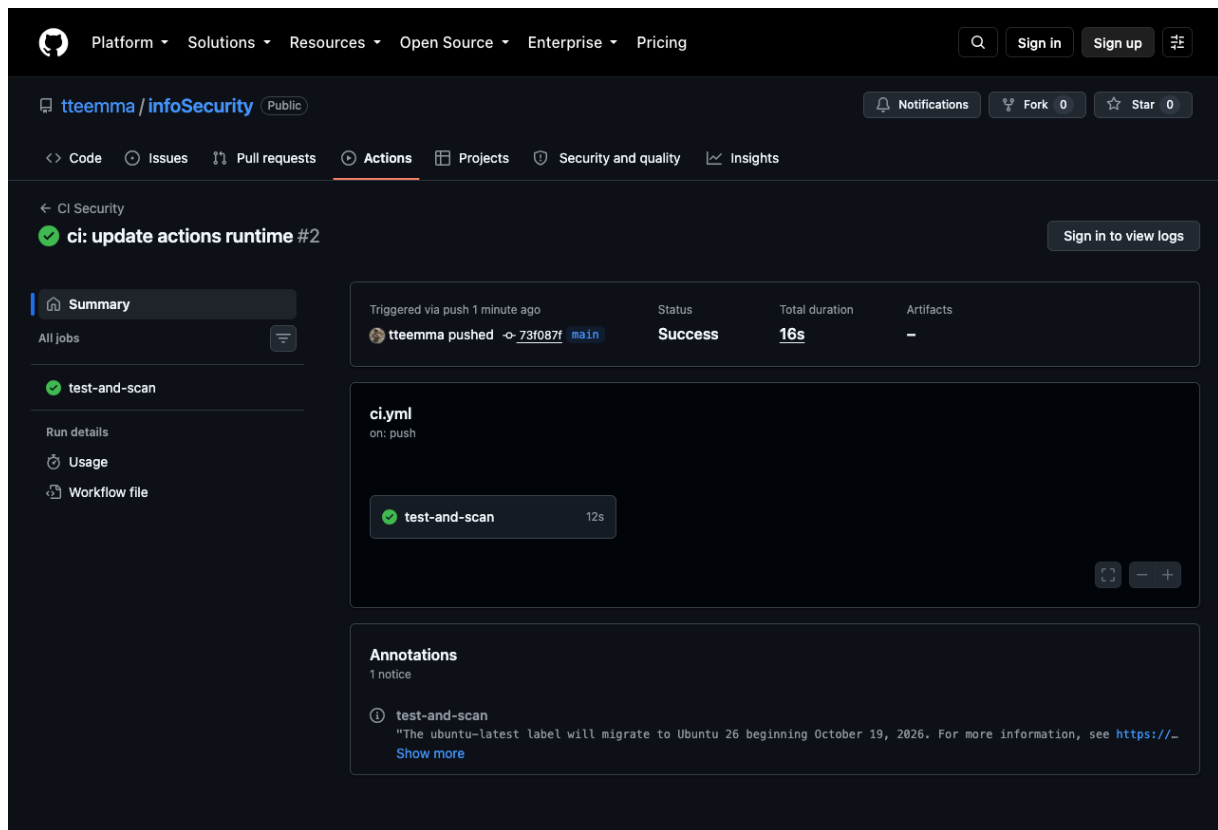


6.2 Результат локального SCA

```
$ npm audit --audit-level=high
found 0 vulnerabilities
```



Те же команды успешно выполнены GitHub Actions в pipeline № 2:



7. Тестирование

Тесты запускаются командой `npm test`. Проверены следующие сценарии:

- успешная аутентификация и выдача JWT;
- отказ при неверном пароле;
- отказ на SQLi-строку вместо логина;
- запрет доступа без JWT и с изменённым JWT;
- корректный HTTP 413 при превышении лимита тела;
- валидация тела создаваемого поста;
- создание и чтение поста;
- экранирование XSS-последовательностей;
- хранение bcrypt-хэша вместо открытого пароля.

Фактический результат локального запуска:

```
tests 11
pass 11
fail 0
```

8. Принципы проектирования

Реализация следует KISS: используются только необходимые сущности. DRY соблюдается за счёт единого JWT middleware, общей валидации и функции безопасного представления поста. Ответственность разделена: репозитории работают с БД, AuthService — с учётными данными и JWT, middleware — с доступом. Зависимости передаются через конструкторы, поэтому части можно заменять и тестировать отдельно. Наследование намеренно не применяется: для данной небольшой модели

композиция проще и яснее.

9. Ссылки

- Публичный репозиторий: <https://github.com/tteemma/infoSecurity>
- Последний успешный pipeline: <https://github.com/tteemma/infoSecurity/actions/runs/35531012781>

10. Вывод

Разработан функциональный REST API из трёх методов. Реализованы параметризованные SQL-запросы, XSS-экранирование, bcrypt-хэширование, JWT и централизованный контроль доступа. Автоматические тесты, SAST и SCA включены в GitHub Actions и локально проходят успешно.